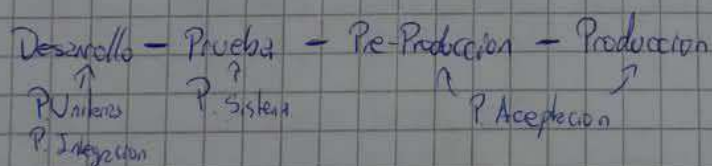


Niveles de Prueba

orden que se ven a probar las cosas

- Pruebas Unitarias → las hace el desarrollador (se hace en la implementación)
 - Pruebas de Integración → las hace el desarrollador o el testing (se hace en la implementación y se manda al testing o directamente en la prueba)
 - Pruebas de ~~Integración~~ Sistema → las realiza los tester (se hace en la prueba)
 - Pruebas de Aceptación → las realiza el usuario (se hace en el despliegue)
- se prueban cada componente, solo componentes individuales, si hay un error se soluciona rápido
pruebas TDD (test driven development)
- pruebas de interfaces, ver si los componentes funcionan relacionándose/integrándose
se prueba un (build)
- prueba de versión, lo hace alguien diferente al que lo construyó y sea objetivo, ver si el sistema en su globalidad opera satisfactoriamente
- pruebas que las hace el usuario

Ambiente para la construcción de Software



Casos de Prueba

un escenario de uso de ejemplo de una funcionalidad del sistema

Set de condiciones o variables bajo las cuales un tester determina si el software está funcionando

Nos ayuda a reproducir defectos.

↳ Normalmente los bugs están en los "bordes"

Derivación de Casos de Prueba (las reglas de negocio son muy importantes)

Refinamiento y Backlog

- Product Backlog (listado de lo que se necesita) - $\rightarrow$ Product Goal (objetivo estratégico por qué?)
- Sprint Backlog (listado de lo que se realiza en el sprint) - $\rightarrow$ Sprint Goal (el objetivo del sprint el por qué técnico)
- Incremento (software funcionando entregado) - $\rightarrow$ Definición de Done (estado de calidad requerido para liberar)

Parcial 2:

Testing

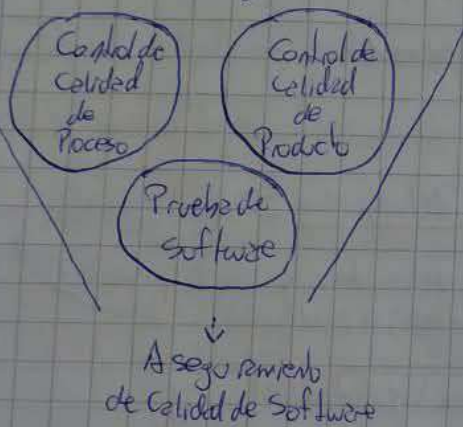
= Proceso destructivo con el objetivo de encontrar **defectos** (si presencia se asume)

Proceso de control: prueban una versión del producto (sus funcionalidades) que compare la versión de código con los Requerimientos (F y NF)

se crean Casos de Prueba y hacen el Testing sistemático

$\rightarrow$ Validar y verificar
Req = función función

Admin de Config



Quality Assurent $\neq$ Testing

Testing exitoso $\rightarrow$ encontrar defectos
(el testing no asegura calidad) $\rightarrow$ el testing hace visible los defectos

* Error vs defecto

- un error es un problema/ que no se trasladó a otra etapa del proceso de desarrollo
 - un defecto es un problema que se trasladó a otra etapa y se descubre (sale más plata)
- defecto $\rightarrow$ produce "FALLA"

↓
mal funcionamiento del software

un defecto puede ocasionar una FALLA o no

• Severidad $\rightarrow$ determina la gravedad del defecto

- 1 - Bloqueante (no te deja hacer nada)
- 2 - Crítico (quien hacer algo y no me deja)
- 3 - Mayor (no puedes hacer algo pero hay un atajo)
- 4 - Menor (te da error pero puedes hacerlo)
- 5 - Cosmético (errores de ortografía/colores/tipos)

• Prioridad $\rightarrow$ define lo necesario que es para el usuario

- 1 - Urgente
- 2 - Alta
- 3 - Medio
- 4 - Baja